

Projektdokumentation

Plant Planner

Software Engineering
Media Systems 4. Semester SoSe24
Prof. Dr. Larissa Putzar

Name

Mat Nr.



Name

Mat Nr.



Name

Mat Nr.



Name

Mat Nr.



Inhaltsverzeichnis

1 Projektidee	1
2 Anforderungen	1
2.1 User Stories	2
2.2 Funktional	4
2.3 Nicht-funktional	5
3 Software-Architektur	5
3.1 3-Schichten-Modell	6
3.1.1 Präsentationsschicht	6
3.1.2 Geschäftslogikschicht	6
3.1.3 Datenschicht	6
3.2 Sicherheitsschicht	6
3.3 Client-Server-Modell	7
3.3.1 Client	7
3.3.2 Server	7
4 Implementation	7
4.1 Entwurfsprinzipien	7
4.1.1 DRY (Don't Repeat Yourself)	7
4.1.2 KISS (Keep It Simple, Stupid)	8
4.1.3 SRP (Single Responsibility Principle)	8
4.1.4 ISP (Interface Segregation Principle)	8
4.1.5 DIP (Dependency Inversion Principle)	8
4.2 Umsetzung	8
4.2.1 Pflanzen aus Pflanzenliste zur eigenen Liste hinzufügen	8
4.2.2 Eigene Pflanzenliste ansehen	9
4.2.3 Pflanze löschen	10
4.2.5 Pflanzenname ändern	10
4.2.4 Pflanzenkalender ansehen	11
5 Testing	11
5.1 Zielsetzung	12
5.2 Unit Tests	12
5.2.1 Arrange, Act, Assert	12
5.2.2 Mocking	12
5.2.2.1 Simulation von Datenbankinteraktionen	13
5.2.2.2 Überprüfung der Benutzer- und Pflanzendaten	13
5.2.2.3 Validierung der Sicherheitsmechanismen	13
Vorgehensmodell	13
6.1 Sprints und Meetings	14
6.2 Rollenverteilung	14
6.3 Kanban-Elemente	14
6.4 Flexibilität und Anpassungen	16

6.5 Zusammenfassung	16
7 Reflexion	17
7.1 Zusammenarbeit	17
7.2 Lessons Learned	17
8 Anhang	18
Abbildung 7: Sequenzdiagramm: Löschen einer Pflanze	18
Abbildung 8: Sequenzdiagramm: Hinzufügen einer Pflanze zum User	18
Abbildung 9: Sequenzdiagramm: Pflanze einen Nickname geben	20
Abbildung 10: Sequenzdiagramm: Pflanze eine Notiz geben	21
Abbildung 11: Sequenzdiagramm: Aktivieren/Deaktivieren E-Mail-Service	22
Abbildung 12: Sequenzdiagramm: User sieht sich Gieß Planung auf dem Kalender an	23
Abbildung 13: Entity Relationship Model	24
Abbildung 14: Use-Case-Diagramm	24
Abbildung 15: Klassendiagramm	25
Abbildung 16: Aktivitätsdiagramm: Home-page	26
9 Abbildungsverzeichnis	26

1 Projektidee

Nach einem Brainstorming hat sich das Team für die Entwicklung einer webbasierten Anwendung entschieden, die Menschen bei der individuellen Pflege ihrer Zimmerpflanzen unterstützt. Das Ziel ist es, eine benutzerfreundliche Plattform zu schaffen, die den Benutzer einen Plan zum Düngen und Gießen ihrer Pflanzen erstellt und diesen innerhalb eines Kalenders darstellt.

Die Idee entstand aus der Notwendigkeit, eine leicht zugängliche Informationsquelle zu bieten, die die Pflanzenpflege vereinfacht. Anstelle einer separaten mobilen App fiel die Entscheidung auf eine reaktionsschnelle Webanwendung, die auf allen Geräten zugänglich ist. Dies senkt die Einstiegshürden für neue Nutzer und erweitert damit die Zielgruppe. Zudem bietet das Projekt die Möglichkeit, Kenntnisse in Java Spring zu vertiefen und den Umgang mit dem Framework zu verbessern.

2 Anforderungen

Der Nutzer soll sich auf der Webseite registrieren oder anmelden können. Nach erfolgreicher Anmeldung soll der Nutzer auf die Home-Seite weitergeleitet werden. Im angemeldeten Zustand soll dem Nutzer auf jeder Seite eine Navigationsleiste zur Verfügung stehen, über die er zu verschiedenen Seiten navigieren oder seinen Account löschen kann.

Die Home-Seite soll einen Kalender anzeigen, der den aktuellen Monat zeigt. An den entsprechenden Tagen sollen Icons angezeigt werden, die darauf hinweisen, wann Pflanzen gegossen oder gedüngt werden müssen. Beim Hovering über die Icons sollen die betroffenen Pflanzen angezeigt werden. Zusätzlich soll der Nutzer über Buttons in der Kalenderansicht vor- und zurückblättern können, um die Pflegeplanung im Voraus sehen zu können.

Die Seite "Pflanzen hinzufügen" soll eine Liste aller Pflanzen enthalten, die der Benutzer zu seiner persönlichen Liste hinzufügen kann. Mit einem Klick auf einen Button soll der Benutzer die gewünschte Pflanze hinzufügen können.

Auf der Seite "Meine Pflanzen" soll der Benutzer die Pflanzen sehen können, die er in seinem Account hinzugefügt hat. Die Pflanzen sollen in einer Liste angezeigt werden, deren Elemente aufklappbar sind, um die Details der Pflanze einzusehen. Neben Informationen wie dem Namen der Pflanze, dem Lieblingsplatz, der Lieblingstemperatur, der Bewässerungs- und Düngerfrequenz sowie der idealen Luftfeuchtigkeit, soll der Nutzer der Pflanze auch einen Spitznamen und eine individuelle Notiz hinzufügen können.

2.1 User Stories

In den User Stories sind die Anforderungen und Funktionen, die der Nutzer benötigt, aus Sicht des Nutzers festgehalten.

2.1.1 Registrierung

Priorität:	Hoch
Story Points:	8
User-Story:	Als User will ich mich registrieren können
Voraussetzungen:	Der User nutzt einen unbenutzten Namen und eine valide E-Mail-Adresse

2.1.2 Anmeldung

Priorität:	Hoch
Story Points:	3
User-Story:	Als User will ich mich anmelden können
Voraussetzungen:	Der User hat sich bereits registriert

2.1.3 Abmeldung

Priorität:	Hoch
Story Points:	2
User-Story:	Als User will ich mich abmelden können
Voraussetzungen:	Der User ist aktuell angemeldet

2.1.4 Account Löschung

Priorität:	Mittel
Story Points:	5
User-Story:	Als User will ich meinen Account löschen können
Voraussetzungen:	Der User ist aktuell angemeldet

2.1.5 Pflanzen auswählen

Priorität:	Hoch
Story Points:	34
User-Story:	Als User will ich Pflanzen auswählen können, die ich besitze
Voraussetzungen:	Der User ist aktuell angemeldet

2.1.6 Pflanzenliste ansehen

Priorität:	Hoch
Story Points::	8
User-Story:	Als User will ich eine Liste aller meiner Pflanzen haben
Voraussetzungen:	Der User ist aktuell angemeldet

2.1.7 Pflanzen aus Pflanzenliste löschen

Priorität:	Mittel
Story Points:	3
User-Story:	Als User möchte ich Pflanzen, die ich nicht mehr habe, aus meiner Liste entfernen können
Voraussetzungen:	Der User ist aktuell angemeldet und hat Pflanzen in seiner Pflanzenliste

2.1.8 Pflanzenkalender

Priorität:	Hoch
Story Points:	34
User-Story:	Als User will ich eine Kalenderansicht haben, auf der die anfallenden Aufgaben der Pflanzen zu sehen sind
Voraussetzungen:	Der User ist aktuell angemeldet und hat Pflanzen in seiner Pflanzenliste

2.1.9 Pflanzennamen in Pflanzenkalender sehen

Priorität:	Hoch
Story Points:	5
User-Story:	Als User will ich in den Kalendereinträgen über die Aktions-Icons hovern und die betroffenen Pflanzen sehen können
Voraussetzungen:	Der User ist aktuell angemeldet und hat Pflanzen in seiner Pflanzenliste

2.1.10 Pflanzen Pflegehinweise

Priorität:	Mittel
Story Points:	5
User-Story:	Als User möchte ich Pflegehinweise zu meinen Pflanzen ansehen können
Voraussetzungen:	Der User ist aktuell angemeldet und hat Pflanzen in seiner Pflanzenliste

2.1.11 Pflanzen benennen

Priorität:	Mittel
Story Points:	3
User-Story:	Als User möchte ich meinen Pflanzen eigene Namen geben können, um gleichartige Pflanzen unterscheiden zu können
Voraussetzungen:	Der User ist aktuell angemeldet und hat Pflanzen in seiner Pflanzenliste

2.1.12 Pflanzen eine Notiz anhängen

Priorität:	Niedrig
Story Points:	3
User-Story:	Als User möchte ich Notizen zu meinen Pflanzen notieren können
Voraussetzungen:	Der User ist aktuell angemeldet und hat Pflanzen in seiner Pflanzenliste

2.1.13 Gieß-Benachrichtigungen zu Pflanzen erhalten

Priorität:	Niedrig
Story Points:	8
User-Story:	Als User möchte ich Benachrichtigungen erhalten wenn ich meine Pflanzen gießen muss
Voraussetzungen:	Der User ist aktuell angemeldet und hat Pflanzen in seiner Pflanzenliste

2.1.14 Benachrichtigungen ein- und ausschalten

Priorität:	Niedrig
Story Points:	2
User-Story:	Als User möchte ich bei den Pflanzen Hinweisen eine Benachrichtigung per Klick aktivieren können
Voraussetzungen:	Der User ist aktuell angemeldet und hat Pflanzen in seiner Pflanzenliste

2.2 Funktional

1. **Registrierung und Anmeldung:** Der Nutzer soll in der Lage sein, einen Account zu erstellen und sich zu einem späteren Zeitpunkt in diesen einzuloggen. Die Daten des Nutzers sollen in seinem Account gespeichert werden, so dass er diese unabhängig vom Endgerät abrufen kann.
2. **Kalender ansehen:** Der Nutzer soll in der Kalenderansicht die Daten ansehen können, an denen Pflanzen gegossen oder gedüngt werden müssen. Des Weiteren soll er erkennen können, um welche Pflanzen es sich handelt. Dazu sollen die Namen und die Spitznamen der Pflanzen angezeigt werden, um Verwechslungsgefahr bei mehreren der gleichen Pflanzen zu vermeiden.

3. **Pflanzen dem Nutzer Account hinzufügen:** In der Pflanzenliste sollen alle Pflanzen enthalten sein, die der Website bekannt sind. Der Nutzer soll in der Lage sein, Pflanzen seinem Account hinzuzufügen.
4. **Pflanzen Details abrufen:** Die Nutzer Pflanzenliste soll alle Pflanzen enthalten, die der Nutzer seiner Liste hinzugefügt hat. Er soll in der Lage sein Details zu allen seinen Pflanzen zu sehen. Die Details sollen enthalten: Name, Idealplatz, Idealtemperatur, Häufigkeit des Gießens und Düngens und ideale Luftfeuchtigkeit.
5. **Spitznamen vergeben und Notizen hinzufügen:** In der Nutzer Pflanzenliste soll es möglich sein, einer Pflanze einen Spitznamen zu geben und Notizen hinzuzufügen.
6. **Löschung eines nicht mehr genutzten Accounts:** Der Nutzer sollte in der Lage sein, einen alten und nicht mehr gebrauchten Account zu löschen.

2.3 Nicht-funktional

1. **Einfache Bedienbarkeit:** Nutzer sollen schnell und einfach in der Lage sein, Pflanzen zu ihrer Liste hinzuzufügen und zu erkennen, wann welche Pflanzen gegossen und gedüngt werden müssen.
2. **Systemunabhängig:** Die Website soll auf möglichst vielen Endgeräten nutzbar sein und die Formatierung soll vollen Zugriff auf die Website erlauben, unabhängig vom Endgerät, das der Nutzer verwendet.
3. **Gute Performanz:** Um eine ideale Nutzererfahrung zu gewährleisten, muss die Website Performant laufen und kurze Ladezeiten haben.
4. **Wartungsfreundlichkeit:** Um einfache und schnelle Wartung auch in der Zukunft zu gewährleisten, muss die Struktur der Website übersichtlich und nachvollziehbar sein.

3 Software-Architektur

Dieses Kapitel erläutert die Architektur des Gardening Planner Projekts, das unter Anwendung des 3-Schichten-Modells und des Client-Server-Modells entwickelt wurde. Die Kombination aus den beiden Modellen bietet eine robuste Grundlage für das Projekt. Zusätzlich wurde die Architektur durch die Sicherheitsschicht erweitert, die die HTTP-Anfragen überprüft und authentifiziert.

Die klare Trennung der Verantwortlichkeiten verbessert die Wartbarkeit und Erweiterbarkeit der Anwendung, während die Client-Server-Architektur eine flexible und skalierbare Lösung

ermöglicht. Diese Architektur unterstützt die kontinuierliche Weiterentwicklung und Anpassung an neue Anforderungen.

3.1 3-Schichten-Modell

Das 3-Schichten-Modell teilt die Anwendung in drei Hauptschichten: Präsentation, Geschäftslogik und Datenzugriff.

3.1.1 Präsentationsschicht

Die Präsentationsschicht ist für die Benutzerinteraktion verantwortlich und wird durch Controller-Klassen wie `CalendarController`, `LoginController` und `PlantListController` repräsentiert. Diese Klassen verarbeiten HTTP-Anfragen und rendern HTML-Ansichten.

3.1.2 Geschäftslogikschicht

Diese Schicht enthält die Geschäftsregeln und Logik der Anwendung. Service-Klassen wie `UserAccountDetailsService` und `CalendarUtil` implementieren die Kerngeschäftslogik, z. B. die Berechnung von Kalenderdaten und die Verwaltung von Benutzerkonten.

3.1.3 Datenschicht

Die Datenschicht greift auf die Datenbank zu und wird durch Repository-Schnittstellen wie `IUserRepository` und `IPlantRepository` dargestellt. Diese Schnittstellen ermöglichen das Abrufen, Ändern, Speichern und Löschen von Datenbankeinträgen.

3.2 Sicherheitsschicht

In dieser Schicht werden die HTTP-Anfragen der Client-Schicht geprüft und entsprechend bearbeitet. Anfragen auf die Login- und Register-Seite und auf öffentlichen Ressourcen werden immer genehmigt, alle anderen Anfragen werden nur für angemeldete Nutzer angenommen. Genehmigte Anfragen werden an die Präsentationsschicht weitergeleitet. Die Funktion dieser Schicht wird von Spring Security implementiert.

3.3 Client-Server-Modell

Das Client-Server-Modell trennt die Frontend- und Backend-Komponenten der Anwendung.

3.3.1 Client

Der Client besteht aus HTML, CSS und Thymeleaf und läuft im Webbrowser des Benutzers. Er sendet HTTP-Anfragen an den Server und empfängt HTML-Seiten als Antworten.

3.3.2 Server

Der Server verarbeitet die Anfragen des Clients, führt die Geschäftslogik aus und greift auf die Datenbank zu. Er besteht aus der Spring Boot Anwendung, die Controller, Services und Repositories enthält. Der Server generiert und liefert HTML-Seiten.

4 Implementation

Das Projekt wurde vollständig in Java unter Verwendung des Spring Frameworks entwickelt. Java wurde aufgrund seiner Robustheit und der umfangreichen Bibliotheksunterstützung gewählt. Es wird eine H2-Datenbank verwendet, die sich für die Entwicklung und das Testen als sehr praktisch erwiesen hat.

Für das Dependency-Management und den Build-Prozess wurden Maven verwendet. Zur Versionskontrolle und für die Zusammenarbeit wurden Git und GitHub genutzt.

4.1 Entwurfsprinzipien

Es wurde versucht, die Entwurfsprinzipien DRY, KISS, SRP, ISP und DIP konsequent anzuwenden.

4.1.1 DRY (Don't Repeat Yourself)

Das DRY-Prinzip wurde umgesetzt, indem wiederverwendbarer Code in gemeinsame Methoden und Klassen ausgelagert wurde. Zum Beispiel wurde die Logik zur Validierung von Benutzereingaben und zur Authentifizierung zentral in den entsprechenden Service-Klassen implementiert, anstatt diese Logik in mehreren Controllern zu duplizieren.

4.1.2 KISS (Keep It Simple, Stupid)

Der Code wurde so einfach und verständlich wie möglich gehalten, indem komplexe Logik vermieden wurde und große Methoden in kleinere, überschaubare Methoden unterteilt wurden.

4.1.3 SRP (Single Responsibility Principle)

Das SRP-Prinzip wurde umgesetzt, indem jede Klasse und Methode nur eine einzige Verantwortlichkeit hat. Beispielsweise ist der "PlantListController" nur für die Verwaltung der Pflanzenlisten-Ansicht zuständig, während die Authentifizierung und Benutzerverwaltung in separaten Klassen organisiert ist.

4.1.4 ISP (Interface Segregation Principle)

Das ISP-Prinzip wurde durch die Definition kleiner, spezialisierter Schnittstellen eingehalten, anstatt großer, allgemeiner Schnittstellen. Dies stellt sicher, dass Klassen nur die Methoden implementieren, die sie tatsächlich benötigen.

4.1.5 DIP (Dependency Inversion Principle)

Das DIP-Prinzip wurde umgesetzt, indem Abhängigkeiten auf abstrakte Schnittstellen anstatt auf konkrete Implementierungen ausgerichtet wurden. Dadurch bleibt der Code flexibel und erweiterbar.

4.2 Umsetzung

Das Projekt bietet eine Vielzahl von Funktionen, die dem Nutzer eine einfache Verwaltung seiner Pflanzen ermöglichen. Hier sind die wichtigsten Funktionen und deren Implementierung:

4.2.1 Pflanzen aus Pflanzenliste zur eigenen Liste hinzufügen

Ein PlantListController wurde erstellt, um die Anzeige aller Pflanzen zu ermöglichen. Die Funktion zum Hinzufügen einer Pflanze zur eigenen Liste wird über ein Formular mit einem POST-Request an den Endpunkt /pflanzeHinzufuegen umgesetzt. Hierbei wird eine neue UserPlant-Instanz erzeugt und in der Datenbank gespeichert.

```
private static final String ADD_PLANT_ENDPOINT = "/pflanzeHinzufuegen";
```

```

@PostMapping(ADD_PLANT_ENDPOINT)
public String addPlant(Model model, Authentication authentication,
@AuthenticationPrincipal UserAccountDetails user, int plantId) {

    var insertPlant = getPlantFromPlantTable(plantId);

    var currentUser = getCurrentUser(user);

    saveNewUserPlant(insertPlant, currentUser);

    return PLANT_LIST_REDIRECT;
}

```

Abbildung 1: Code Ausschnitt PlantListController (addPlant)

Frontend: Ein Formular wird für jede Pflanze in der Liste angezeigt, welche dem Nutzer ermöglicht, durch Klicken auf einen Button die Pflanze zur eigenen Liste hinzuzufügen.

4.2.2 Eigene Pflanzenliste ansehen

Der UserPlantsController verwaltet die Pflanzen des Nutzers. Ein GET-Request an den Endpunkt /benutzerPflanzen liefert eine Liste der Pflanzen, die dem aktuellen Nutzer gehören. Diese Liste wird aus der Datenbank abgerufen und an die View übergeben.

Frontend: Die Pflanzen des Nutzers werden in einer übersichtlichen Liste angezeigt. Jedes Listenelement kann durch Klicken einen Klick erweitert werden, um zusätzliche Informationen zu der Pflanze anzuzeigen.

▼ Monstera deliciosa

Spitzname: 

Pflanzenart: Monstera deliciosa

Lieblingsplatz: Heller Standort, keine direkte Sonne

Lieblingstemperatur: 18.0°C

Durst:

Sommer: 4x monatlich

Winter: 2x monatlich

Düngen:

Sommer: 2x monatlich

Winter: 1x monatlich

Luftfeuchtigkeit: 60%

Notizen:





Abbildung 2: Ausschnitt Pflanze des Nutzers

4.2.3 Pflanze löschen

Ebenfalls im UserPlantsController wird ein POST-Request an den Endpunkt /pflanzeLoeschen verarbeitet. Hierbei wird die entsprechende Pflanze anhand ihrer ID aus der Datenbank entfernt.

```

@PostMapping(USER_PLANTS_DELETION_ENDPOINT)
public String deletePlant(Model model, int userPlantId) {
    var plantDeletion = getUserPlant(userPlantId);
    try {
        iUserPlantRepository.deleteById(plantDeletion.getId());
    } catch (Exception e) {
        model.addAttribute(attributeName:"error", e.getMessage());
    }
    return REDIRECT + USER_PLANTS_ENDPOINT;
}

```

Abbildung 3: Code Ausschnitt UserPlantController (deletePlant)

Frontend: Für jede Pflanze in der Liste gibt es einen Lösch-Button, der das Löschen der Pflanze ermöglicht.

4.2.5 Pflanzenname ändern

Im UserPlantsController wird ein POST-Request an den Endpunkt /pflanzeumbenennen verarbeitet, um den Namen einer Pflanze zu ändern. Die neue Bezeichnung wird in der Datenbank aktualisiert.

```

@PostMapping(USER_PLANT_CHANGE_NICKNAME)
public String changeNickname(Model model, int userPlantId, String nickname){
    var plant = getUserPlant(userPlantId);
    try {
        iUserPlantRepository.updateNicknameById(nickname, plant.getId());
    } catch (Exception e) {
        model.addAttribute(attributeName:"error", e.getMessage());
    }
    return REDIRECT + USER_PLANTS_ENDPOINT;
}

```

Abbildung 4: Code Ausschnitt UserPlantController (changeNickname)

Frontend: Jede Pflanze in der Liste hat ein Formularfeld zur Namensänderung, das direkt im Interface bearbeitet und gespeichert werden kann.

4.2.4 Pflanzenkalender ansehen

Der CalendarController stellt die Funktionalität zum Anzeigen des Pflanzenkalenders bereit. Ein GET-Request an den Endpunkt /kalender liefert die Kalenderdaten für den aktuellen Monat und optional für zukünftige Monate.

Die Klasse CalendarUtil berechnet die Bewässerungs- und Düngetermine für den Kalender. Die darin enthaltene Methode calculateDates verwendet das Hinzufügedatum der Pflanze, das aktuelle Datum sowie die saisonalen Pflegeintervalle (Sommer und Winter). Sie

berechnet, in welchen Abständen eine Pflanze gegossen oder gedüngt werden muss und erstellt daraus eine Liste von Terminen.

Frontend: Der Kalender wird als Tabelle dargestellt, in der die Tage des Monats sowie Symbole für Bewässerungs- und Düngetermine angezeigt werden. Der Nutzer kann durch verschiedene Monate navigieren. Die berechneten Termine werden im Kalender als Symbole angezeigt, sodass der Nutzer leicht erkennen kann, wann seine Pflanzen gegossen oder gedüngt werden müssen.

5 Testing

Das Testen ist ein essentieller Bestandteil des Softwareentwicklungsprozesses, um sicherzustellen, dass die Anwendung korrekt funktioniert und robust ist. In diesem Kapitel wird die Vorgehensweise beim Schreiben und Durchführen von Tests für die Anwendung beschrieben. Der Fokus liegt dabei auf Unit-Tests, die wesentliche Funktionalitäten der Anwendung abdecken.

5.1 Zielsetzung

Die Hauptziele der Tests in diesem Projekt sind:

- Sicherstellung der Korrektheit der Anwendungslogik
- Validierung der korrekten Interaktion zwischen den Schichten der Anwendung
- Überprüfung der Sicherheitsfunktionen
- Gewährleistung der Datenintegrität in der Datenbank

5.2 Unit Tests

Unit-Tests sind eine fundamentale Methode zur Sicherstellung der Codequalität und -stabilität. Sie fokussieren sich auf das Testen einzelner Komponenten der Anwendung in Isolation, typischerweise Methoden oder kleine Klassen.

In diesem Projekt wurden Unit-Tests intensiv genutzt, um zentrale Funktionen der Anwendung zu validieren. Hierbei wurden Mocking-Frameworks wie Mockito verwendet, um Abhängigkeiten zu simulieren und isolierte Tests zu ermöglichen.

5.2.1 Arrange, Act, Assert

Durch die konsequente Anwendung des Arrange, Act, Assert-Konzepts in den Unit-Tests wurde eine klare und nachvollziehbare Teststruktur geschaffen. Dies erleichtert nicht nur das Verständnis und die Wartung der Tests, sondern stellt auch sicher, dass die Anwendungslogik robust und fehlerfrei funktioniert.

5.2.2 Mocking

Mocking ist ein wichtiger Bestandteil der Unit-Tests in diesem Projekt. Es ermöglicht das Simulieren von Objekten und deren Verhalten, ohne deren tatsächliche Implementierungen zu verwenden.

Im Folgenden werden die Bereiche beschrieben, in denen Mocking in diesem Projekt hauptsächlich eingesetzt wurde.

5.2.2.1 Simulation von Datenbankinteraktionen

Durch Mocking der Repositorys konnten die Datenbankzugriffe isoliert und die Geschäftslogik unabhängig von der Datenbankschicht getestet werden.

5.2.2.2 Überprüfung der Benutzer- und Pflanzendaten

Mocks wurden verwendet, um Benutzer- und Pflanzendaten zu simulieren und zu überprüfen, ob die Geschäftslogik korrekt darauf zugreift.

5.2.2.3 Validierung der Sicherheitsmechanismen

Durch Mocking der Sicherheitskomponenten konnten Authentifizierungs- und Autorisierungsprozesse getestet werden, ohne tatsächliche Benutzeranmeldungen durchführen zu müssen.

6 Vorgehensmodell

Unser Vorgehensmodell orientiert sich an den Prinzipien des agilen Frameworks Scrum, das Flexibilität und iterative Prozesse anwendet. Im Laufe des Projekts haben wir zwar hauptsächlich Scrum verwendet, jedoch auch Teile von Kanban integriert, die unsere spezifischen Anforderungen und Arbeitsweisen besser widerspiegeln.

6.1 Sprints und Meetings

Wir haben unsere Arbeit in einwöchige Sprints unterteilt. Jeder Sprint begann montags mit einem Treffen online über Discord, bei dem der Fortschritt des vorherigen Sprints überprüft wurde und der Plan für die kommende Woche erstellt wurde. Dabei wurde besprochen, was gut lief, welche Herausforderungen aufgetreten sind und welche Aufgaben im nächsten Sprint erledigt werden sollten. Diese regelmäßigen Besprechungen sind ein Kernbestandteil von Scrum, bekannt als Sprint Planning und Daily Scrum Meetings. Auch wenn die Treffen nicht täglich stattfanden, halfen sie, einen guten Überblick über das Projekt und den Fortschritt zu erhalten.

6.2 Rollenverteilung

Eine klare Rollenverteilung, wie sie in klassischem Scrum vorgesehen ist (Product Owner, Scrum Master, Entwicklungsteam), war in dem Team nicht vorhanden. Stattdessen haben alle gemeinsam an den verschiedenen Aufgaben gearbeitet. Dies weicht zwar von der traditionellen Scrum-Methodik ab, hat jedoch für ein kleines Team gut funktioniert, da jeder flexibel war und so unterschiedliche Rollen kennenlernen konnte.

6.3 Kanban-Elemente

Neben den Scrum-Elementen wurden auch Elemente aus Kanban integriert. Kanban fokussiert sich auf die Visualisierung von Arbeitsprozessen und kontinuierliche Verbesserung. Durch die Verwendung von Miro konnten ein Kanban-ähnliches System geschaffen werden, das half, den Arbeitsfluss zu managen und die Aufgabenverteilung zu optimieren. Die Aufgaben der jeweiligen Sprints wurden mit Hilfe des Boards in "To-Do", "Doing" und "Done" aufgeteilt und zusätzlich mit den Anfangsbuchstaben der Gruppenmitglieder markiert, um einen guten Überblick zu behalten.

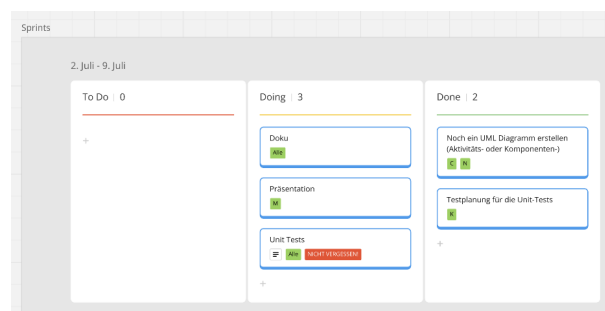


Abbildung 5: Miro Board Sprintplanung

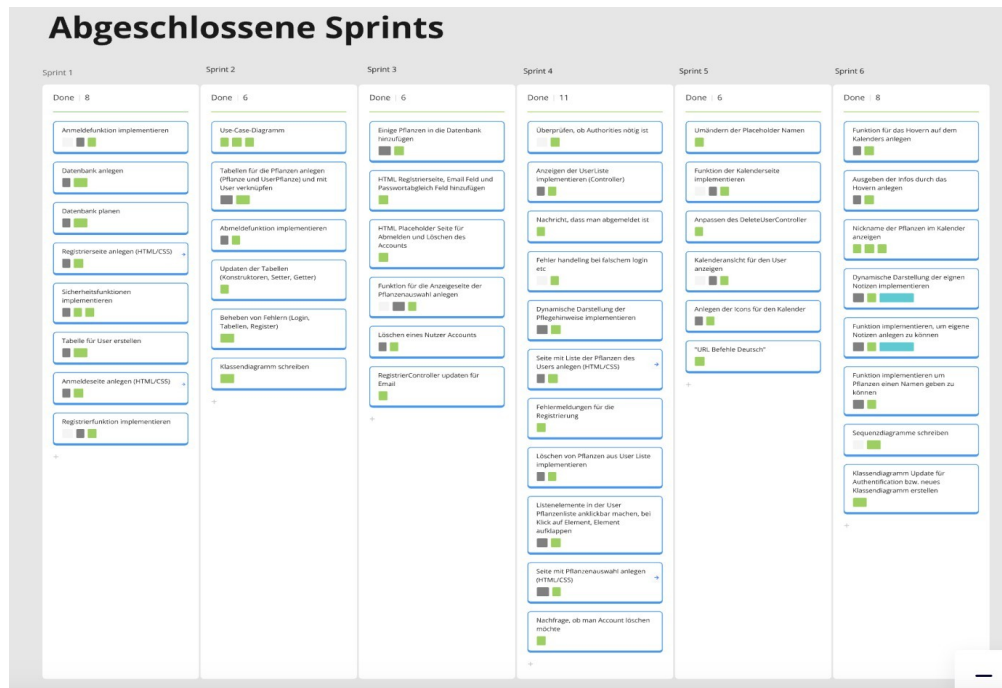


Abbildung 6: Miro Board Sprint Übersicht

6.4 Flexibilität und Anpassungen

Ein entscheidendes Merkmal des Vorgehensmodells war die Flexibilität, die es ermöglichte, Prozesse kontinuierlich zu überprüfen und anzupassen. Regelmäßige Retrospektiven halfen dabei, die Arbeitsweise zu reflektieren und Verbesserungen für zukünftige Sprints zu identifizieren. Diese agilen Methoden ermöglichten es, effizient auf Veränderungen und neue Anforderungen zu reagieren. So wurde beispielsweise in einigen Sprints die Entscheidung getroffen, die Sprint Länge von einer auf zwei Wochen zu verlängern, um gesetzte Sprint-Ziele im angemessenen Zeitrahmen zu erreichen.

6.5 Zusammenfassung

Durch die Einteilung in einwöchige Sprints, regelmäßige Meetings zur Planung und Überprüfung sowie die gemeinsame Bearbeitung von Aufgaben konnten die Prinzipien agiler Entwicklung erfolgreich umsetzen. Dabei wurde sich nicht strikt an eine Methode gehalten, sondern das Beste aus beiden Vorgehensmodellen integriert, um den spezifischen Bedürfnissen unseres Projekts gerecht zu werden.

Dieses hybride Vorgehen half dabei, die Vorteile beider Methoden zu kombinieren: die Struktur von Scrum sowie die Transparenz und Flexibilität von Kanban.

7 Reflexion

7.1 Zusammenarbeit

Die Zusammenarbeit in der Gruppe funktionierte ohne nennenswerte Probleme. Sprint Ziele konnten größtenteils in der geplanten Zeit erreicht werden und die Tätigkeiten der einzelnen Teammitglieder variierten von Sprint zu Sprint.

Es wurden regelmäßige Meetings abgehalten, um den Fortschritt zu besprechen und eventuelle Hindernisse gemeinsam zu lösen. Diese Meetings trugen erheblich zur Klarheit und Effizienz bei, da alle Mitglieder dadurch über den aktuellen Stand informiert wurden und gemeinsam Strategien zum weiteren Vorgehen entwickeln konnten. Die Kommunikation dafür erfolgte hauptsächlich über Discord und das Miro Board. Falls ein Problem bei einzelnen Aufgaben eines Gruppenmitgliedes auftraten, wurde dafür extra Zeit geschaffen, um es als Team zu lösen.

Auch die Stundeneinteilung zwischen den Gruppenmitgliedern erfolgte fair und gleichmäßig.

7.2 Lessons Learned

Rückblickend gibt es zwei Bereiche, in denen Verbesserungen möglich sind. Ein besserer Umgang mit der Versionskontrolle, beispielsweise mithilfe von Branches bei GitHub, hätte dazu beigetragen, Merge-Konflikte zu erleichtern und zu reduzieren.

Weiterhin wäre es hilfreich gewesen, sich vor Projektbeginn intensiver mit dem verwendeten Framework auseinanderzusetzen. Ein besseres Verständnis des Frameworks hätte dazu beigetragen, Probleme wie beispielsweise beim Login-Prozess zu vermeiden. Eine

gründlichere Vorbereitung hätte uns ermöglicht, derartige Herausforderungen leichter bewältigen zu können.

8 Anhang

Abbildung 7: Sequenzdiagramm: Löschen einer Pflanze

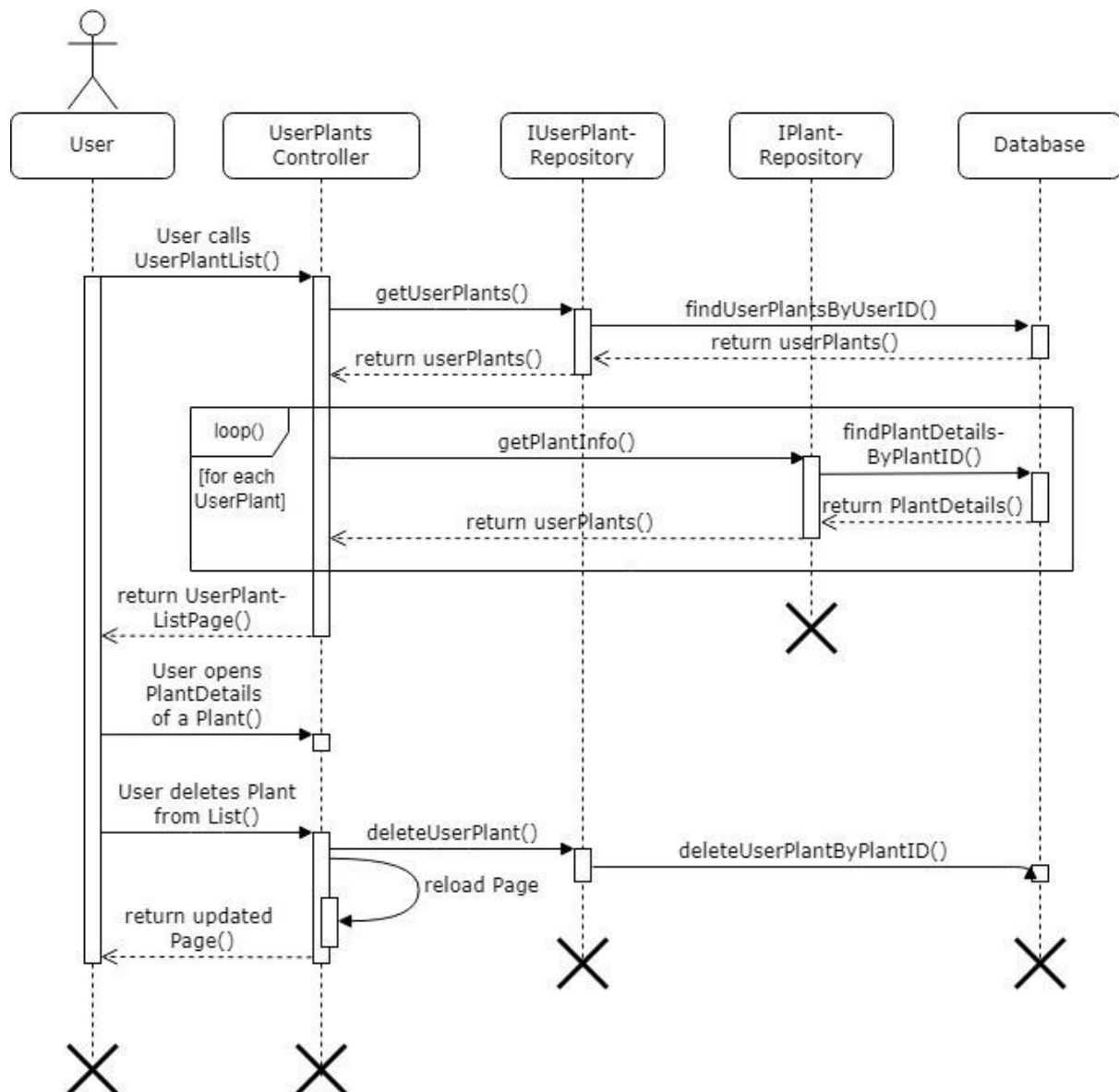


Abbildung 8: Sequenzdiagramm: Hinzufügen einer Pflanze zum User

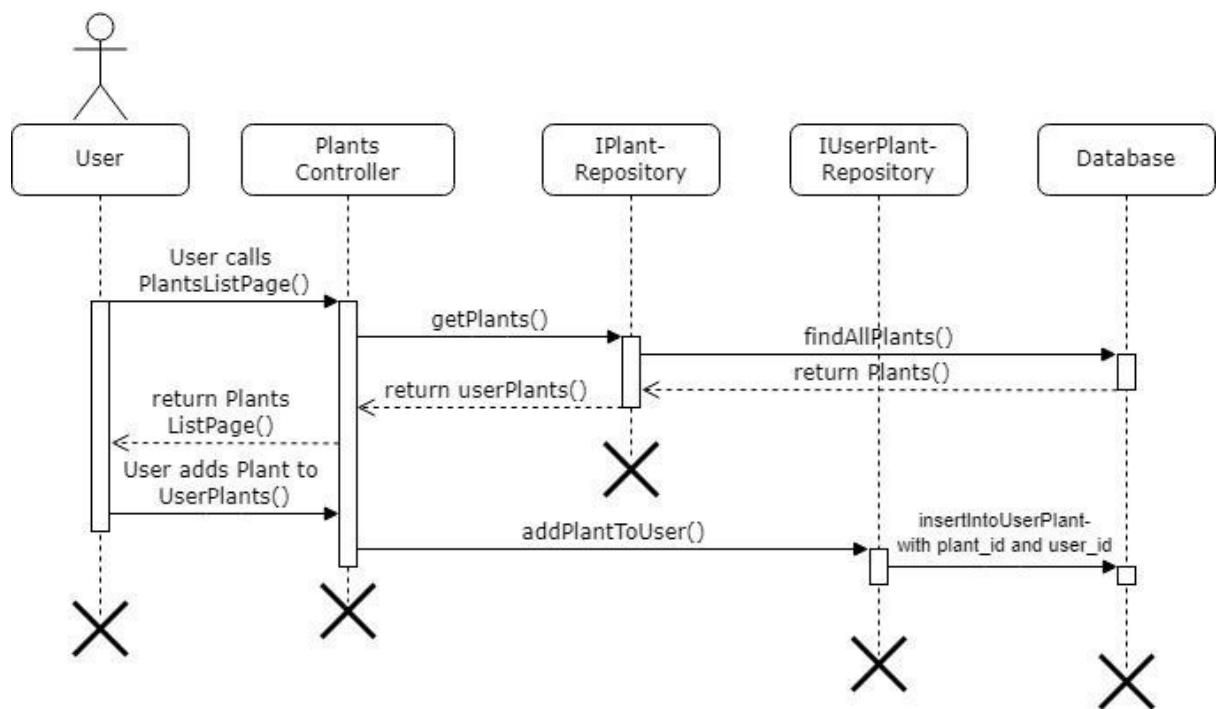


Abbildung 9: Sequenzdiagramm: Pflanze einen Nickname geben

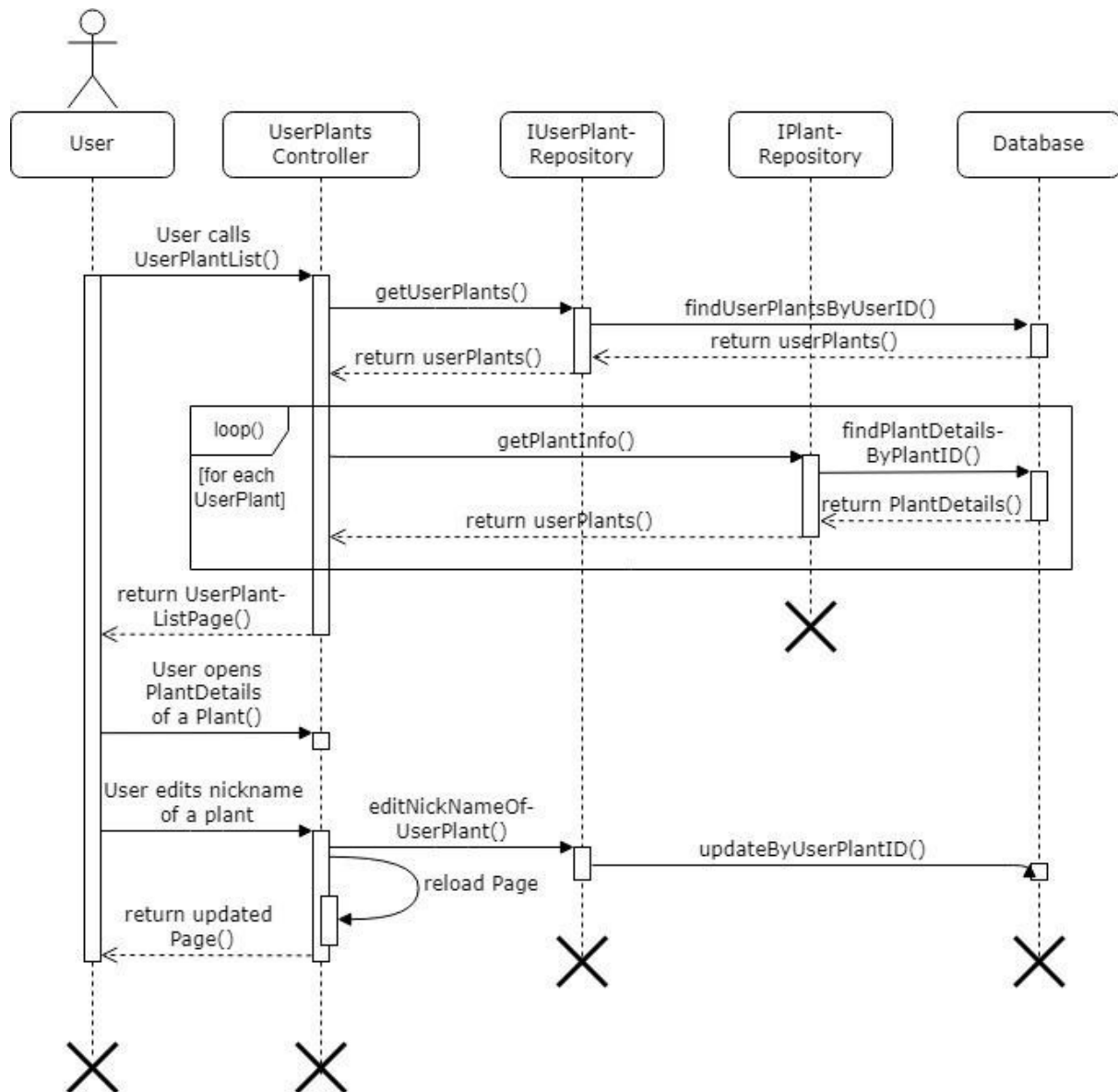


Abbildung 10: Sequenzdiagramm: Pflanze eine Notiz geben

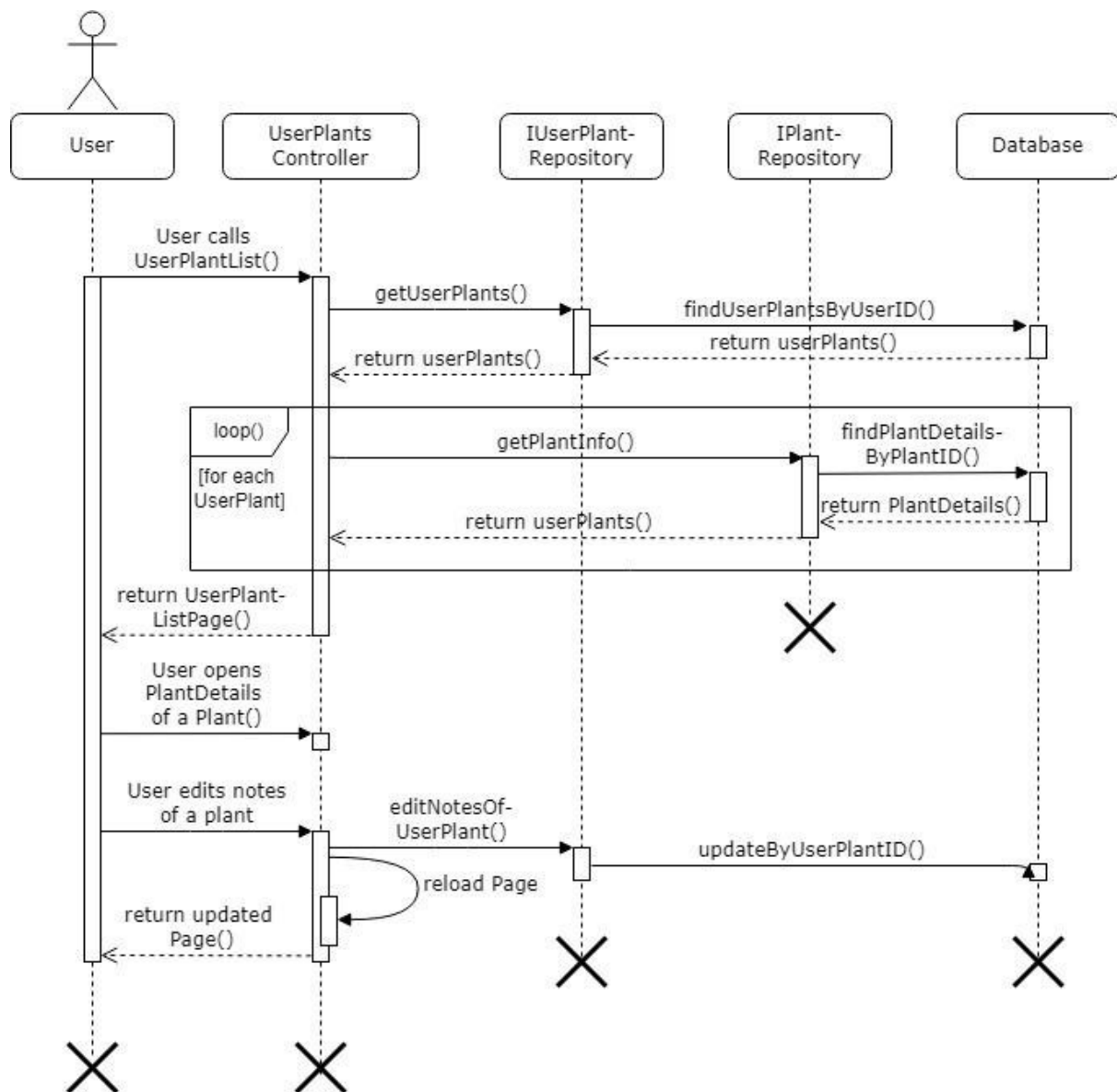


Abbildung 11: Sequenzdiagramm: Aktivieren/Deaktivieren E-Mail-Service

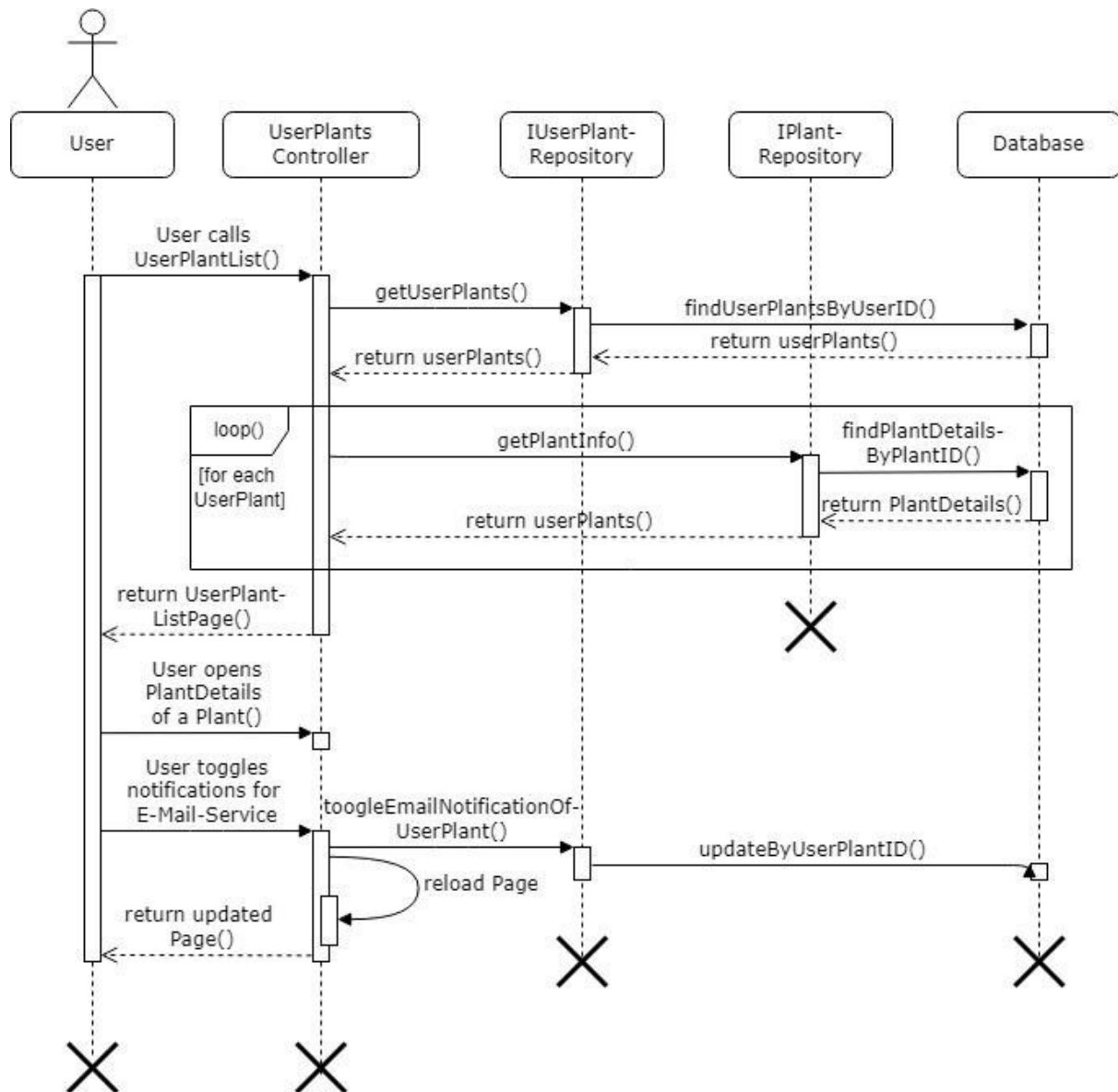


Abbildung 12: Sequenzdiagramm: User sieht sich Gieß
Planung auf dem Kalender an

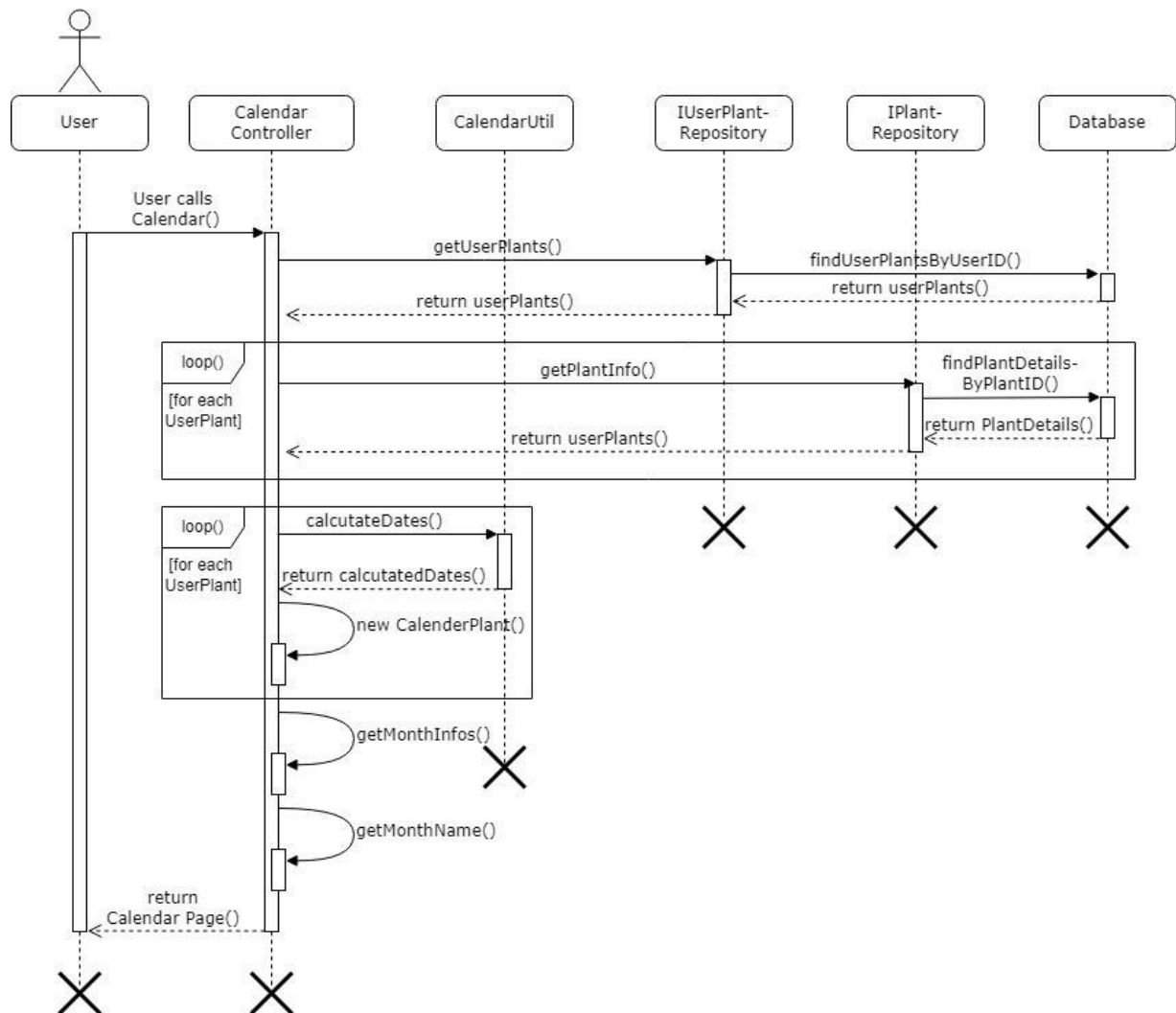


Abbildung 13: Entity Relationship Model

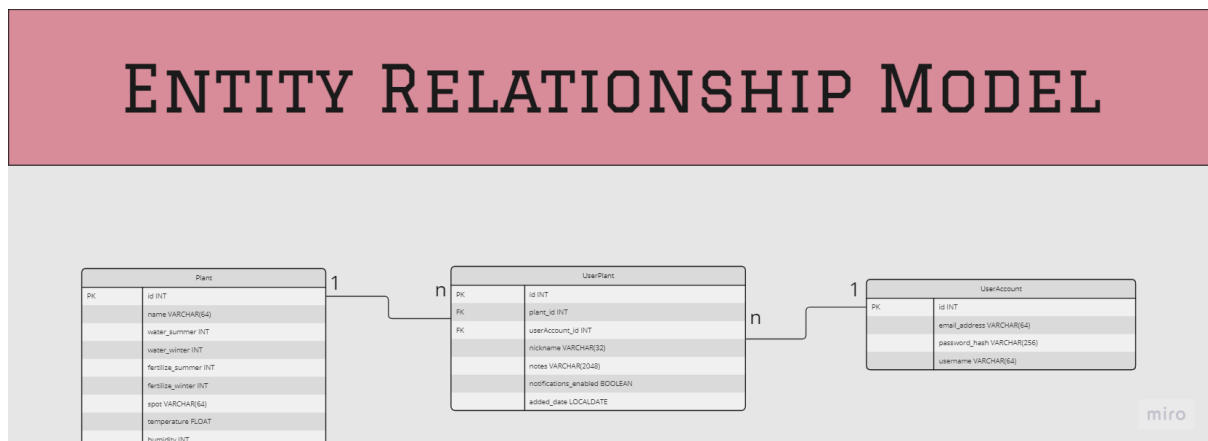


Abbildung 14: Use-Case-Diagramm

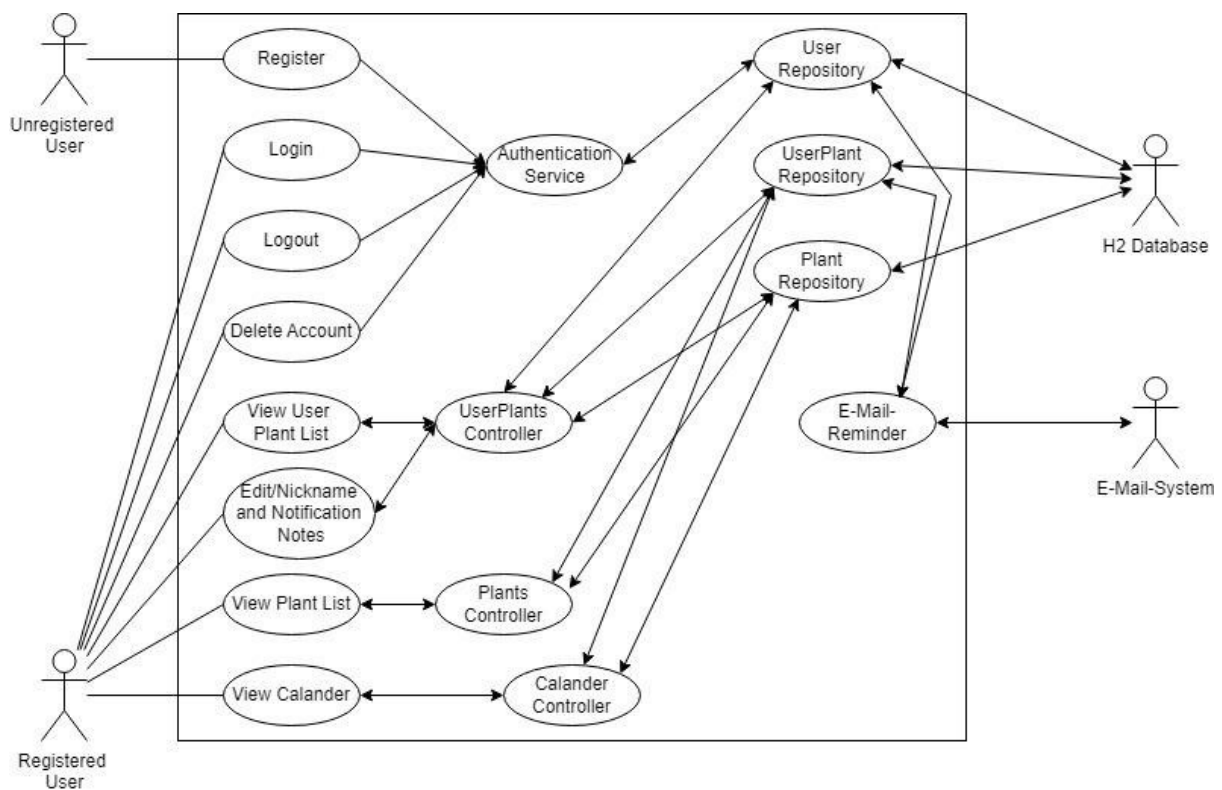


Abbildung 15: Klassendiagramm

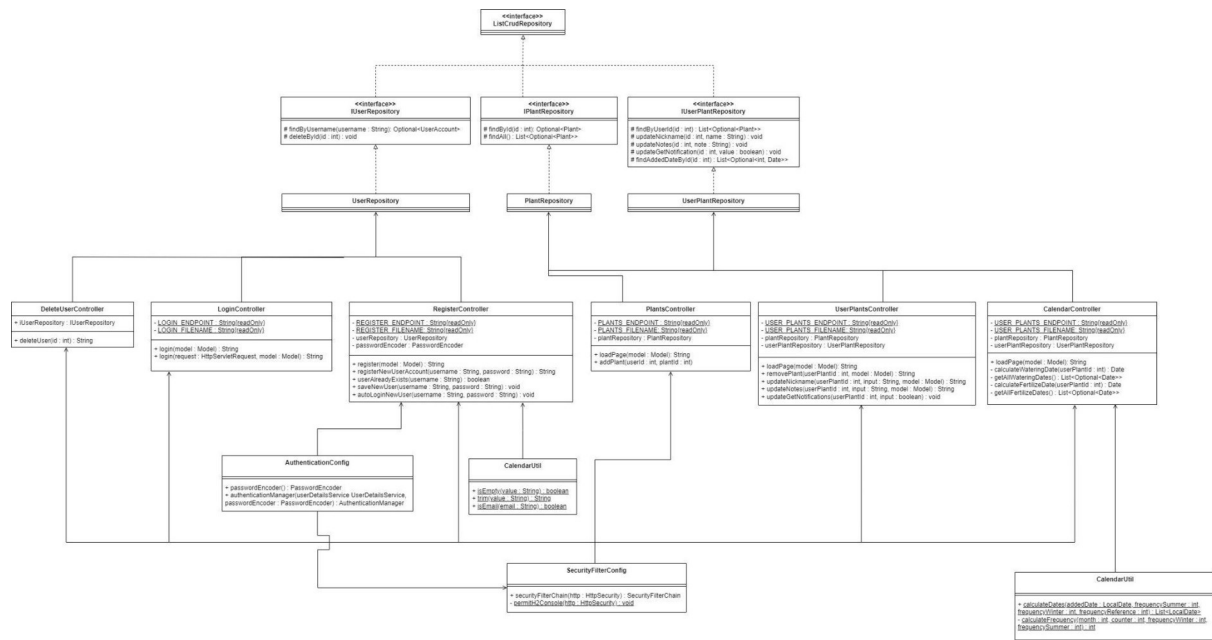
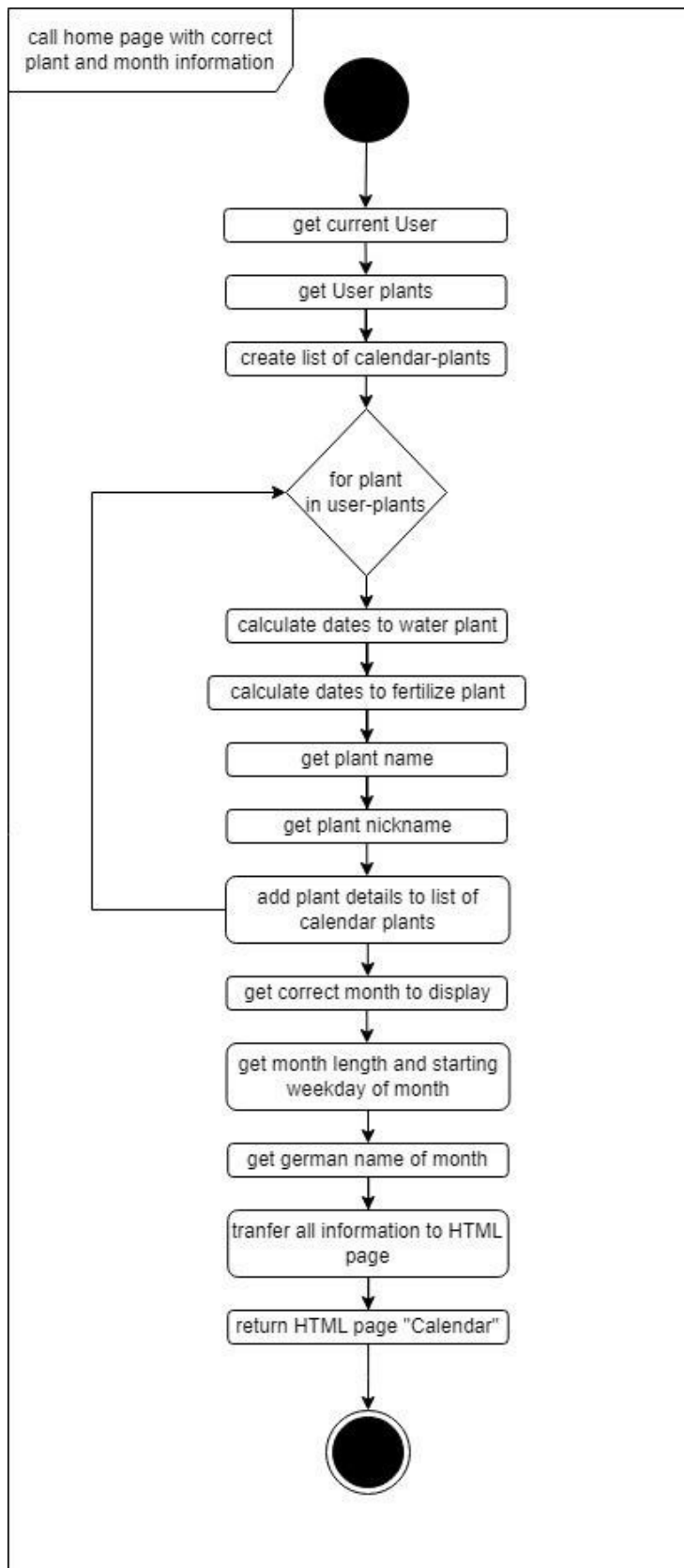


Abbildung 16: Aktivitätsdiagramm: Home-page



9 Abbildungsverzeichnis

Abbildung 1: Code Ausschnitt PlantListController (addPlant)	9
Abbildung 2: Ausschnitt Pflanze des Nutzers	9
Abbildung 3: Code Ausschnitt UserPlantsController (deletePlant)	10
Abbildung 4: Code Ausschnitt UserPlantsController (changeNickname)	10
Abbildung 5: Miro Board Sprintplanung	14
Abbildung 6: Miro Board Sprint Übersicht	14
Abbildung 7: Sequenzdiagramm: Löschen einer Pflanze	17
Abbildung 8: Sequenzdiagramm: Hinzufügen einer Pflanze zum User	18
Abbildung 9: Sequenzdiagramm: Pflanze eine Nickname geben	19
Abbildung 10: Sequenzdiagramm: Pflanze eine Notiz geben	20
Abbildung 11: Sequenzdiagramm: Aktivieren/Deaktivieren E-Mail-Service	21
Abbildung 12: Sequenzdiagramm: User sieht sich Gieß Planung auf dem Kalender an	22
Abbildung 13: Entity Relationship Model	23
Abbildung 14: Use-Case-Diagramm	23
Abbildung 15: Klassendiagramm	24
Abbildung 16: Aktivitätsdiagramm: Home-Page	25